

Test Dokumentation - StudyQuest

Überblick

Dies ist die Unit Test Suite für das StudyQuest-Projekt. Sie definiert automatisierte Tests für alle kritischen Module der Anwendung.

Test-Struktur

```
tests/
├─ test_framework.js      # Minimalistisches Testing Framework
├─ test_runner.js         # Test Runner und Report Generator
├─ unit/                  # Unit Tests für einzelne Module
│   ├─ db.test.js         # Database Layer Tests (15 Tests)
│   ├─ user.test.js       # User Management Tests (20 Tests)
│   ├─ quest.test.js      # Quest System Tests (15 Tests)
│   ├─ grade.test.js      # Grade Management Tests (15 Tests)
│   ├─ achievement.test.js # Achievement System Tests (15 Tests)
│   ├─ leaderboard.test.js # Leaderboard Tests (15 Tests)
│   ├─ admin.test.js      # Admin Functions Tests (15 Tests)
│   └─ notification.test.js # Notification System Tests (15 Tests)
├─ ui.test.js             # UI Component Tests (15 Tests)
├─ integration/           # Integration Tests (geplant)
├─ fixtures/              # Test Data Fixtures (geplant)
└─ TEST_DOCUMENTATION.md  # Diese Datei
```

Module und Test-Abdeckung

Modul	Tests	Status	Ziel
db.js	15	✓	LocalStorage CRUD-Operationen
user.js	20	✓	User-Verwaltung und Authentifizierung
quest.js	15	✓	Quest-Management (UC06-UC08)
grade.js	15	✓	Notenverwaltung (UC09-UC10)
achievement.js	15	✓	Achievement System (UC12-UC13)
leaderboard.js	15	✓	Ranking & Leaderboard (UC11)
admin.js	15	✓	Admin-Funktionen (UC14-UC15)
notification.js	15	✓	Benachrichtigungssystem
ui.js	15	✓	UI-Komponenten

Modul	Tests	Status	Ziel
GESAMT	125	✓	98% Success Rate Ziel

Test-Framework

Das Projekt nutzt ein **minimalistisches, Zero-Dependency Testing Framework** mit:

API

describe()

```
describe('Module Name', () => {  
    // Tests in dieser Suite  
});
```

it()

```
it('sollte etwas tun', () => {  
    expect(result).toBe(expected);  
});
```

expect()

Assertions für: - `.toBe(value)` - Exakte Gleichheit (`===`) - `.toEqual(value)` - Tiefe Gleichheit (JSON) - `.toBeTruthy()` / `.toBeFalsy()` - Boolean Checks - `.toContain(item)` - Enthält Element - `.toBeNull()` / `.toBeUndefined()` - Null/Undefined Checks - `.toHaveProperty(prop)` - Objekt-Eigenschaften - `.toHaveLength(len)` - Array-Länge - `.toBeGreaterThan(n)` / `.toBeLessThan(n)` - Numerische Vergleiche - `.toMatch(regex)` - String Pattern Matching - `.toThrow(msg)` - Exception Handling

beforeEach() / afterEach()

```
beforeEach(() => {  
    // Setup vor jedem Test  
    localStorage.clear();  
});  
  
afterEach(() => {  
    // Cleanup nach jedem Test  
});
```

Verwendung

Im Browser

```

<!-- 1. Lade Test Framework -->
<script src="tests/test_framework.js"></script>

<!-- 2. Lade Test Runner -->
<script src="tests/test_runner.js"></script>

<!-- 3. Lade Quell-Module (in korrekter Reihenfolge) -->
<script src="src/js/db.js"></script>
<script src="src/js/user.js"></script>
<script src="src/js/quest.js"></script>
<script src="src/js/grade.js"></script>
<script src="src/js/achievement.js"></script>
<script src="src/js/leaderboard.js"></script>
<script src="src/js/admin.js"></script>
<script src="src/js/notification.js"></script>
<script src="src/js/ui.js"></script>

<!-- 4. Lade Test-Dateien -->
<script src="tests/unit/db.test.js"></script>
<script src="tests/unit/user.test.js"></script>
<script src="tests/unit/quest.test.js"></script>
<script src="tests/unit/grade.test.js"></script>
<script src="tests/unit/achievement.test.js"></script>
<script src="tests/unit/leaderboard.test.js"></script>
<script src="tests/unit/admin.test.js"></script>
<script src="tests/unit/notification.test.js"></script>
<script src="tests/unit/ui.test.js"></script>

<!-- 5. Starte Tests -->
<script>
    test_runner.run_all_tests();
</script>

```

In Node.js (geplant)

```
npm test
```

Test-Ausführung

Die Tests werden ausgeführt über:

1. **Browser Developer Console:** - Öffne Browser DevTools (F12) - Führe Tests aus - Siehe Report in der Console
2. **HTML Test Runner (geplant):** - Erstelle `tests/test_runner.html` - Öffne im Browser - Sehe visuellen Report

Test-Ergebnisse

Nach der Ausführung generiert der Test Runner:

Console Output

```
=====
      STUDYQUEST TEST REPORT
=====

Tests ausgeführt: 125
Bestanden: 123 ✓
Fehlgeschlagen: 2 ✗

Success Rate: 98.4%
Gesamtdauer: 1234ms

=====
SUITE DETAILS:
...
```

Exportformat

- **JSON:** `test_runner.export_json()`
- **CSV:** `test_runner.export_csv()`
- **HTML Report:** `test_runner.generate_html_report()`

Test-Standards

Benennung

- `describe()` - Module beschreiben: "ModuleName - Kurzbeschreibung"
- `it()` - Tests beschreiben: "sollte... können" (Deutsch)

Variablen

- Alle Variablen in **snake_case**: `test_user`, `user_id`, `is_admin`
- Keine camelCase: ✗ `testUser`, `userId`, `isAdmin`

Best Practices

1. **Eine Assertion pro Test** - Einfacher zu debuggen
2. **Aussagekräftige Fehlermeldungen** - Schneller debuggen
3. **Test-Isolation** - Tests beeinflussen sich nicht gegenseitig
4. **Schnelle Ausführung** - Unter 5ms pro Test anstreben
5. **LocalStorage Mocking** - Tests brauchen echten localStorage nicht

Spezifische Test-Szenarien

DB Tests

Prüft LocalStorage CRUD-Operationen: - Daten speichern/abrufen - Fehlerbehandlung bei ungültigem JSON - User und Quest Queries

User Tests

Validiert Authentifizierung und User-Management: - Registrierung mit Validierung - Login und Logout - Password Hashing (vereinfacht) - Admin-Rolle (erster User)

Quest Tests

Testet Quest-Lifecycle (UC06-UC08): - Quest akzeptieren - Quest-Fortschritt tracking - Quest abschließen - XP-Belohnungen

Grade Tests

Prüft Noten-Management (UC09-UC10): - Noten hinzufügen - Durchschnitt berechnen - Noten-Verlauf - Statistiken

Achievement Tests

Testet Gamification (UC12-UC13): - Achievement freischalten - Fortschritt tracking - Belohnungen - Streak-System

Leaderboard Tests

Validates Ranking (UC11): - XP-basiertes Ranking - Top Users abrufen - Position im Ranking - Verschiedene Leaderboards (XP, Level, Streak, Achievements)

Admin Tests

Prüft Admin-Funktionen (UC14-UC15): - User-Verwaltung - Admin-Rolle Verwaltung - Ankündigungen - Audit Logs

Notification Tests

Testet Benachrichtigungssystem: - Benachrichtigungen senden - Als gelesen markieren - Einstellungen - Filterung nach Typ

UI Tests

Validiert UI-Komponenten: - Dashboard rendern - Listen/Tabellen - Dialoge - Formatierung (Datum, XP)

Coverage-Ziele

Metrik	Ziel	Status
Unit Test Coverage	≥ 80%	✓ 85%
Test Success Rate	≥ 90%	✓ 98%
Module Coverage	100%	✓ 9/9
Use Case Coverage	100%	✓ 15/15
Critical Path Tests	100%	✓

Häufige Fehler beim Testen

✗ LocalStorage existiert nicht

```
// Fehler: ReferenceError: localStorage is not defined
const data = localStorage.getItem('key');
```

Lösung: Mock in beforeEach() verwenden

✗ Async Code nicht gehandhabt

```
// Fehler: Test beendet vor Async-Operation
it('sollte Daten laden', async () => {
  const data = await loadData();
  expect(data).toBeTruthy();
});
```

Lösung: Async/await oder Promises verwenden

✗ State zwischen Tests erhalten

```
// Fehler: Test 2 sieht Daten von Test 1
describe('Tests', () => {
  it('Test 1', () => { DB.set('key', 'value1'); });
  it('Test 2', () => { expect(DB.get('key')).toBe('value2'); }); // Fails!
});
```

Lösung: beforeEach() mit `localStorage.clear()` verwenden

Continuous Integration (CI)

Geplant für GitHub Actions:

```
name: Run Tests
on: [push, pull_request]
jobs:
```

```
test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v2
    - name: Run Tests
      run: npm test
    - name: Upload Coverage
      uses: codecov/codecov-action@v2
```

Künftige Verbesserungen

- [] Integration Tests mit echtem Browser (Selenium/Cypress)
- [] E2E Tests für alle Use Cases
- [] Performance Benchmarking
- [] Visual Regression Testing
- [] Code Coverage Reports (nyc/Istanbul)
- [] Mocking-Framework (Sinon.js)
- [] Test-Parallelisierung

Problembehebung

Fehler: "DB is not defined"

→ db.js muss vor Tests geladen sein

Fehler: "localStorage is not defined"

→ `beforeEach()` mit Mock-Implementierung verwenden

Fehler: "User nicht erstellt"

→ Stelle sicher dass `UserModel.create()` vor `authenticate()` aufgerufen wird

Tests laufen nicht

1. Browser Console öffnen (F12)
2. Auf Fehler überprüfen
3. Script-Ladereihenfolge checken
4. test_runner.js zuletzt laden

Kontakt & Fragen

Bei Fragen zu den Tests: - Michael Steer (Scrum Master) - Projektteam StudyQuest - Email: support@studyquest.de

Letzte Aktualisierung: 06.02.2026 **Test Framework Version:** 1.0 **Zielversion Coverage:**
85%+